Stage 3 实验报告

于越洋

1 实验内容

Step 7

我在 `frontend/typecheck/namer.py` 中补全 `visitCondExpr` 函数，该函数接受一个条件表达式、一个条件成立时执行的表达式和条件不成立时执行的表达式，用于处理问号表达式。
接下来，我在 `frontend/tacgen/tacgen.py` 中也补全了 `visitCondExpr` 函数，该函数用于生成问号表达式对应的 TAC，生成 TAC 的结构为

```
[tac of condition]
if (condition is false) branch skipLabel

[tac of code when condition is true]
branch exitLabel

skipLabel:
[tac of code when condition is false]

exitLabel:
[tac of code after condition expression]
```

Step 8

首先，我修改了 `frontend/lexer/lex.py` ，在保留字定义中添加了 `continue` 和 `for` 关键字，并在 `frontend/parser/ply_parser.py` 中添加了两者对应的语法规则，以便正常完成词法分析和语法分析。
接下来，我修改了 `frontend/ast/tree.py` ，增加了 `For` 和 `Continue` 语法树节点的定义，并在 `frontend/ast/visitor.py` 中添加了对应的访问函数。
为保证 `continue` 和 `break` 语句正常运作，我修改了 `frontend/scope/scope.py` 中 `Scope` 类的定义，添加了 `loopDepth` 成员变量来记录循环层数，也修改了 `frontend/scope/scopestack.py` 中的 `ScopeStack` 类，添加了对于循环层数的处理，以及 `openLoop` ， `closeLoop` ， `getLoopDepth` 函数用于处理循环作用域。
此后，我补全了 `frontend/typecheck/namer.py` 中的 `visitContinue` 和 `visitFor` 函数。 `visitContinue` 函数只检查是否在循环中，而 `visitFor` 函数处理循环作用域，并特别处理 `for` 循环中止条件不存在的情况，将其赋值为常量 `1` ，即循环会一直执行。同时我还修改了 `visitWhile` 函数，也添加了对于循环作用域的处理。

最后，我在 `frontend/tacgen/tacgen.py` 中添加了 `visitFor` 函数，负责生成 `for` 循环语句对应的 TAC，生成 TAC 的结构为

```
[tac of loop init expression]
branch beginLabel

loopLabel:
[tac of loop update expression]

beginLabel:
[tac of loop condition expression]
if (condition is false) branch breakLabel
[tac of loop body code]
branch loopLabel

breakLabel:
[tac of code after for expression]
```

我还添加了 `visitContinue` 函数，负责生成 `continue` 语句对应的 TAC，即直接跳转到 `continueLabel`。

2 思考题

Step 7

Question 1

> 我们的实验框架里是如何处理悬吊 else 问题的？请简要描述。

Answer：实验框架的 `frontend/parser/ply_parser.py` 中，给出了两种 `statement` 的定义，分别为 `statement_matched` 和 `statement_unmatched`，用于表示 `if-else` 匹配的语句和只有 `if` 没有 `else` 的语句，对应的产生式集合为

```
statement_matched -> if (expression) statement_matched else
statement_matched
statement_unmatched -> if (expression) statement_matched else
statement_unmatched
statement_unmatched -> if (expression) statement
```

可见 `statement_matched` 无法产生 `statement_unmatched`，所以在对具有悬吊 `else` 情况的代码进行语法分析时，一定会先匹配掉所有的 `statement_unmatched`，再匹配 `stament_matched`，也就是 `else` 优先和最近的 `if` 匹配，从而解决悬吊 `else` 问题。

Question 2

> 在实验要求的语义规范中，条件表达式存在短路现象。即：

```
int main() {
    int a = 0;
    int b = 1 ? 1 : (a = 2);
    return a;
}
```

> 会返回 0 而不是 2。如果要求条件表达式不短路，在你的实现中该做何种修改？简述你的思路。

Answer：只需将 `frontend/tacgen/tacgen.py` 中 `visitCondExpr` 函数中的语句 `expr.otherwise.accept(self, mv)` 放在紧随 `expr.then.accept(self, mv)` 之后（语句 `mv.emitBranch(exitLabel)` 之前）的位置，即可不进行短路赋值。

Step 8

Question 1

> 将循环语句翻译成 IR 有许多可行的翻译方法，例如 while 循环可以有以下两种翻译方式：
> 第一种（即实验指导中的翻译方式）：
>
> - `label BEGINLOOP_LABEL`：开始下一轮迭代
> - `cond 的 IR`
> - `beqz BREAK_LABEL`：条件不满足就终止循环
> - `body 的 IR`
> - `label CONTINUE_LABEL`：continue 跳到这
> - `br BEGINLOOP_LABEL`：本轮迭代完成
> - `label BREAK_LABEL`：条件不满足，或者 break 语句都会跳到这儿
>
> 第二种：
>
> - `cond 的 IR`
> - `beqz BREAK_LABEL`：条件不满足就终止循环
> - `label BEGINLOOP_LABEL`：开始下一轮迭代
> - `body 的 IR`
> - `label CONTINUE_LABEL`：continue 跳到这
> - `cond 的 IR`
> - `bnez BEGINLOOP_LABEL`：本轮迭代完成，条件满足时进行下一次迭代
> - `label BREAK_LABEL`：条件不满足，或者 break 语句都会跳到这儿
>
> 从执行的指令的条数这个角度（`label` 不算做指令，假设循环体至少执行了一次），请评价这两种翻译方式哪一种更好？

Answer：第二种翻译方式更好，因为第一种方式每次循环都会固定执行 4 条指令，而第二种方式只有第一次循环会执行 5 条指令，后面每次循环都只执行 3 条指令，所以从执行的指令的条数角度来看，第二种翻译方式执行的总指令条数一般少于第一种翻译方式，因此更好。

Question 2

> 我们目前的 TAC IR 中条件分支指令采用了单分支目标（标签）的设计，即该指令的操作数中只有一个是标签；如果相应的分支条件不满足，则执行流会继续向下执行。在其它 IR 中存在双目标分支（标签）的条件分支指令，其形式如 `br cond, false_target, true_target` 其中 `cond` 是一个临时变量， `false_target` 和 `true_target` 是标签。其语义为：如果 `cond` 的值为0（假），则跳转到 `false_target` 处；若 `cond` 非0（真），则跳转到 `true_target` 处。它与我们的条件分支指令的区别在于执行流总是会跳转到两个标签中的一个。
> 你认为中间表示的哪种条件分支指令设计（单目标 vs 双目标）更合理？为什么？（言之有理即可）

Answer：我认为双目标更合理，因为双目标 TAC 的结构可以改为

```
br condition, skipLabel, thenLabel

thenLabel:
[tac of code when condition is true]
branch exitLabel

skipLabel:
[tac of code when condition is false]

exitLabel:
[tac of code after condition expression]
```

结构更加清晰，阅读时能够快速分辨条件为 `true` 和 `false` 时之星的下一步操作，易于理解。